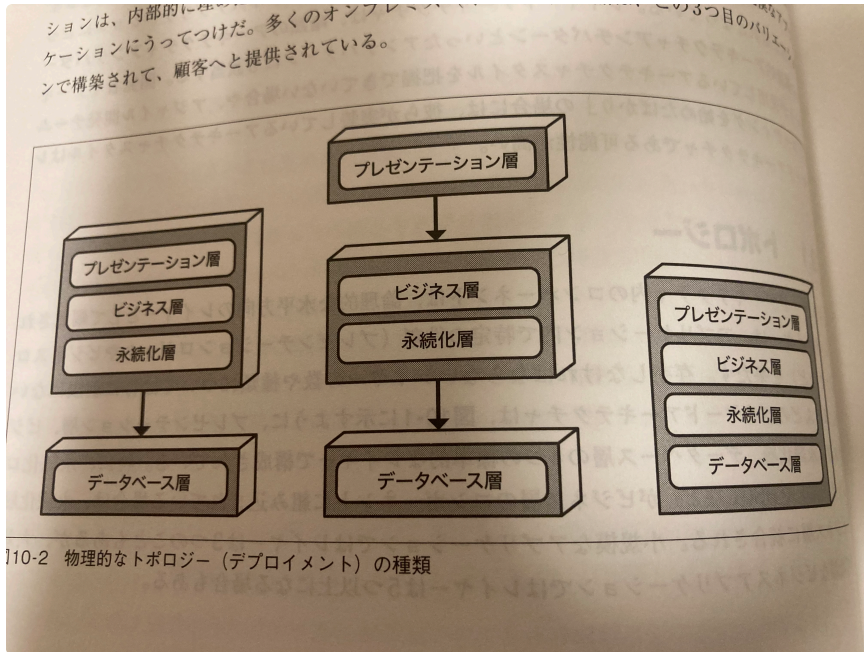


# レイヤードアーキテクチャ

## トポロジー

- 水平方向のレイヤーが複数積み重なった構造
- 一つのレイヤーが一つの役割を果たす
- 標準的な構造では、プレゼンテーション層、ビジネス層、永続化層、データベース層の4層から成る
- トポロジーは色々あり。データベース層を独立させるケースが多い



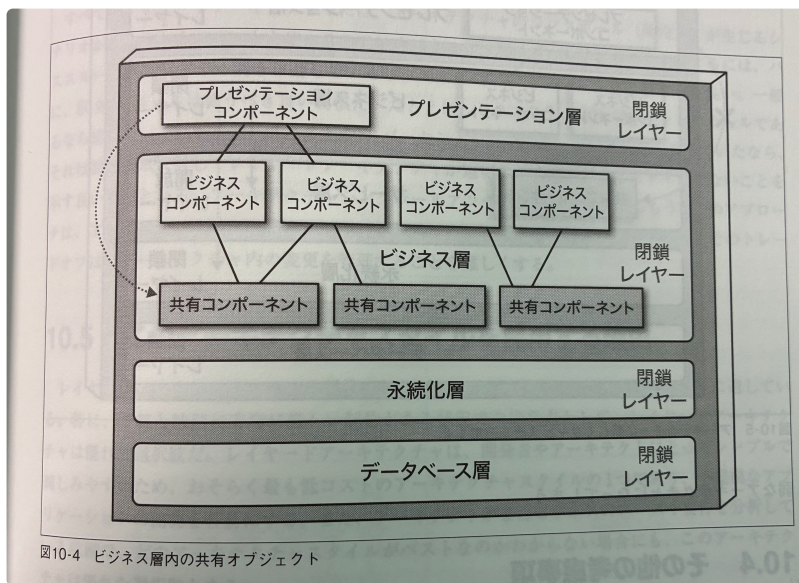
- 技術によって分割されたアーキテクチャである
  - Cons: ビジネスドメインが各層に散らばるため、ドメインの変更が困難、アジリティ欠如
  - Pros: 技術による分割によって**関心事の分離**が実現される

## 層の分離

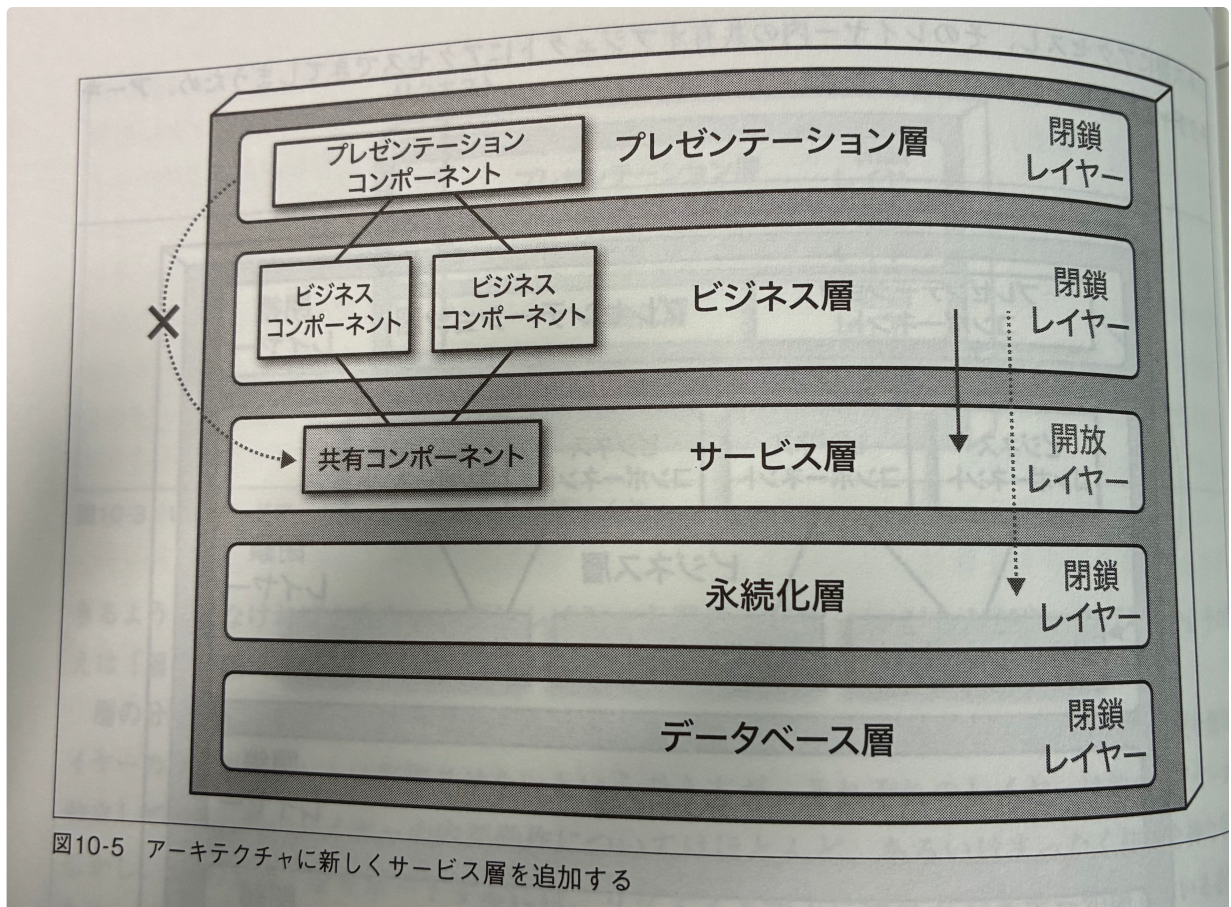
- あるレイヤーの変更を他のレイヤーに影響させない、という考え方。これに従うと**変更しやすいアーキテクチャになる**
- これを実現するには、各レイヤーが**閉鎖レイヤー**である必要がある。つまり、リクエストがどのレイヤーもスキップすることなく、直下のレイヤーを経由しなければならない

## レイヤーの追加

- 前項の通り、**基本的には閉鎖レイヤーが推奨だが、開放レイヤーが理に叶う場合もある**
- 例えば、複数のビジネスコンポーネントから使用できる共通機能 (**共有コンポーネント**) を層に切り出す場合、その層は開放レイヤーにすることが望ましい
  - 共有コンポーネントは、ビジネスコンポーネントから参照できるようにするため、ビジネス層に置く必要がある



- このようにするとプレゼンテーション層から層を跨いだ参照が可能になってしまい、依存性の管理が難しくなる
- 共有コンポーネントを**サービス層**として切り出すことで、プレゼンテーション層からの直接参照を防止し、アーキテクチャの秩序を保つことができる
- ここで、永続化層はビジネス層から直接参照できるよう、開放レイヤーとしておく必要がある



## その他の考慮事項

- ほとんどの場合で、レイヤードアーキテクチャが最適な出発点となる
  - ただし、マイクロサービスなど、他のアーキテクチャに移行しやすい構造を保っておくこと：
    - 再利用を最小限に抑える
    - オブジェクト階層を浅くする

- 適切なモジュール性を維持する
- **アーキテクチャシンクホール**の多発に注意
  - 各層が、他の層から受け取ったリクエストに対して、何の処理もせずに他の層に受け渡すこと
  - 不必要に層を行き来することになり、メモリ消費とパフォーマンスに悪影響を及ぼす

## このアーキテクチャスタイルを採用する理由

- 小規模でシンプルなアプリケーションやWebサイトに適する
- 親しみやすく、低コストなアーキテクチャスタイルである
- 規模が大きくなるにつれ、アジリティやデプロイ容易性が悪化するため、他のよりモジュール化されたアーキテクチャスタイルを採用・移行すべき

## アーキテクチャ特性の評価

| アーキテクチャ特性 | 星評価   | 理由                                           |
|-----------|-------|----------------------------------------------|
| 分割タイプ     | 技術    |                                              |
| 量子数       | 1     | <a href="#">アーキテクチャ量子</a> を参照                |
| デプロイ容易性   | ★     | 変更のたびに全体を再デプロイする必要があるため                      |
| 弾力性       | ★     |                                              |
| 進化性       | ★     |                                              |
| 耐障害性      | ★     | 特定の層が障害を起こすと全ての層が影響するため                      |
| モジュール性    | ★     |                                              |
| 全体的なコスト   | ★★★★★ | シンプルで理解しやすいため                                |
| パフォーマンス   | ★★    |                                              |
| 信頼性       | ★★★   | ネットワークトラフィックに関する課題が少ないため                     |
| スケーラビリティ  | ★     | 特定の機能だけをスケーリングすることが難しいため (スケーリングに限界がある)      |
| シンプルさ     | ★★★★★ |                                              |
| テスト容易性    | ★★    | 単一の層の変更であっても他の全ての層に影響する可能性があり、テストに必要な工数が高いため |